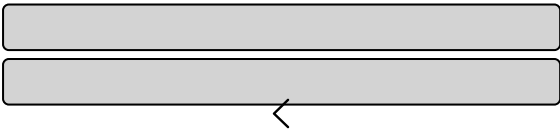


[View in English](#)



- [Apple Developer](#)

- - [Get Started](#)



Explore Get Started

- [Overview](#)
- [Learn](#)
- [Apple Developer Program](#)

Stay Updated

- [Latest News](#)
- [Hello Developer](#)

- - [Platforms](#)



Explore Platforms

- [Apple Platforms](#)
- [iOS](#)
- [iPadOS](#)
- [macOS](#)
- [tvOS](#)

- [visionOS](#)
- [watchOS](#)
- [App Store](#)

Featured

- [Design](#)
- [Distribution](#)
- [Games](#)
- [Accessories](#)
- [Web](#)
- [Home](#)
- [CarPlay](#)

- [Technologies](#)



Explore Technologies

- [Overview](#)
- [Xcode](#)
- [Swift](#)
- [SwiftUI](#)

Featured

- [Accessibility](#)
- [App Intents](#)
- [Apple Intelligence](#)
- [Games](#)
- [Machine Learning & AI](#)
- [Security](#)
- [Xcode Cloud](#)

- [Community](#)

- 



Explore Community

- [Overview](#)
- [Meet with Apple events](#)
- [Community-driven events](#)
- [Developer Forums](#)
- [Open Source](#)

Featured

- [WWDC](#)
- [Swift Student Challenge](#)
- [Developer Stories](#)
- [App Store Awards](#)
- [Apple Design Awards](#)
- [Apple Developer Centers](#)

- 
[Documentation](#)

- 



Explore Documentation

- [Documentation Library](#)
- [Technology Overviews](#)
- [Sample Code](#)
- [Human Interface Guidelines](#)
- [Videos](#)

Release Notes

- [Featured Updates](#)
- [iOS](#)

- [iPadOS](#)
- [macOS](#)
- [watchOS](#)
- [visionOS](#)
- [tvOS](#)
- [Xcode](#)



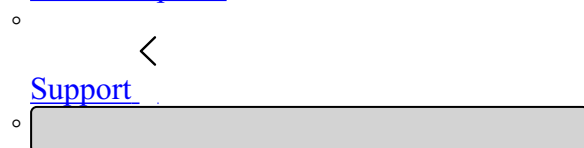
[Downloads](#)

Explore Downloads

- [All Downloads](#)
- [Operating Systems](#)
- [Applications](#)
- [Design Resources](#)

Featured

- [Xcode](#)
- [TestFlight](#)
- [Fonts](#)
- [SF Symbols](#)
- [Icon Composer](#)



[Support](#)

Explore Support

- [Overview](#)
- [Help Guides](#)
- [Developer Forums](#)
- [Feedback Assistant](#)

- [Contact Us](#)

Featured

- [Account Help](#)
- [App Review Guidelines](#)
- [App Store Connect Help](#)
- [Upcoming Requirements](#)
- [Agreements and Guidelines](#)
- [System Status](#)

•



globalnav



Quick Links

- → [Events](#)
- → [News](#)
- → [Forums](#)
- → [Sample Code](#)
- → [Videos](#)

•



=

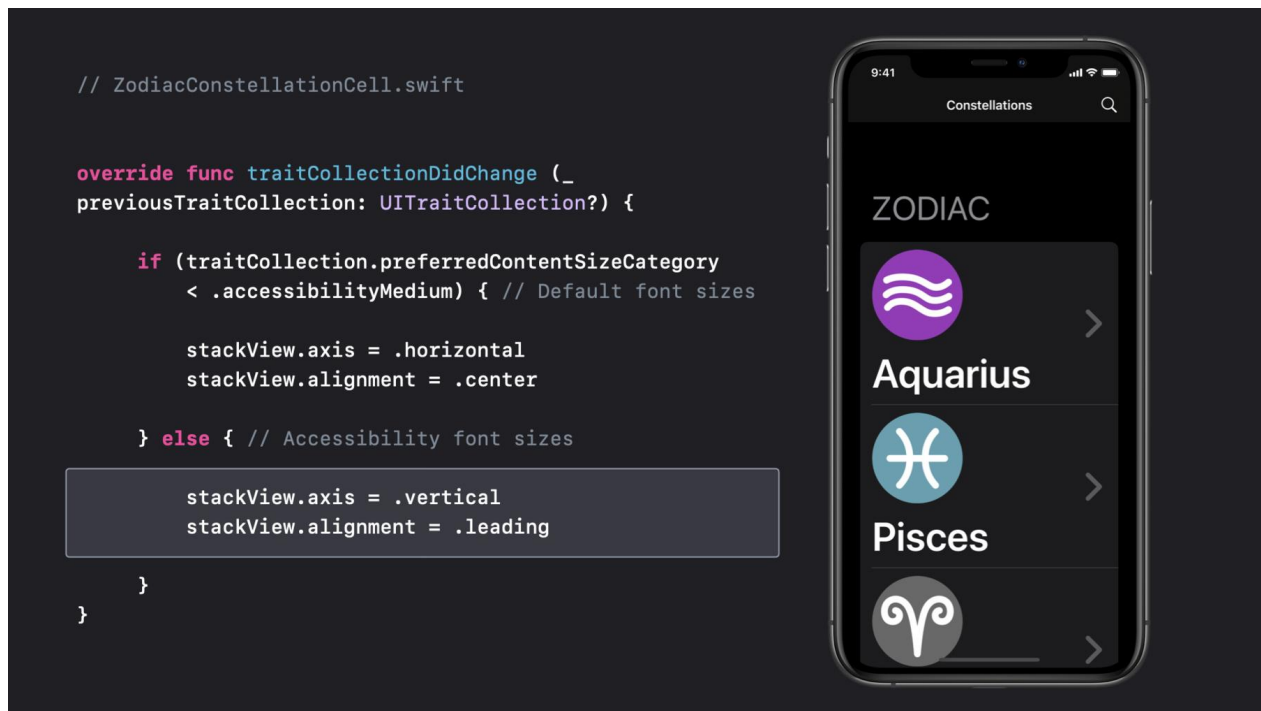


[Videos](#)

[Open Menu](#) [Close Menu](#)

- [Collections](#)
- [Topics](#)
- [All Videos](#)
- [About](#)

[More Videos](#)



- [About](#)
- [Transcript](#)
- [Code](#)

• Make your app visually accessible

When you design with accessibility in mind, you empower everyone to use your app. Discover how to create an adaptive interface for your app that takes a thoughtful approach to color, provides readable text, and accommodates other visual settings to maintain a great experience throughout.

We've designed this session like our user interfaces — to be accessible to all. If you'd like to learn even more about accessibility and design, you may also enjoy “Visual Design and Accessibility,” “Accessibility Inspector,” “Building Apps with Dynamic Type,” and “Introducing SF Symbols.”

Resources

- [Accessibility for UIKit](#)
- - [HD Video](#)
 - [SD Video](#)

Related Videos

WWDC23

- [Create accessible spatial experiences](#)

WWDC21

- [Create accessible experiences for watchOS](#)

WWDC19

- [Accessibility Inspector](#)
- [Introducing SF Symbols](#)
- [Visual Design and Accessibility](#)



Search this video...

Hello, and welcome to WWDC.

Hi, everyone. My name is Drew Haas, and I'm an engineer on the accessibility team. Today, I'm going to talk about how you can make your app visually accessible.

When you think about visual accessibility, you might think about VoiceOver, the built-in screen reader on iOS. And you'd be right, because VoiceOver speaks the items on-screen.

It also allows you to navigate and interact with your device without you needing to look at it. But there's so much more to visual accessibility.

Vision loss is best described as a broad continuum. That's because people experience a wide range of visual impairment. Some people have full sight, partial sight or low sight, and some have no sight at all. And there's a lot in between, like color blindness, light sensitivity, motion sensitivity and more. Since each person's experience with vision loss is unique, iOS offers a range of visual accessibility settings so that everyone can get the most out of their device in a way that works best for them. Today, I'll talk about some of the great ways that your app can provide a visual experience that is inclusive to everyone.

First, I'll talk about the use of color and shapes in your app, which are a great way for you to add a personal touch, like branding.

Next, I want to break down what it means for text to be readable. You can leverage different text attributes and layout changes in your app to make sure everyone has a great reading experience. And last, in accessibility settings, users can customize their display preferences, which may alter the appearance of your app. I'll make sure you know just what to look out for as you make your app visually accessible. When designing an app, sometimes you may ask yourself, "How can I make this particular UI element stand out?" Using color is often a great way to create emphasis, but we don't want to rely on color alone. That's because for people that are color-blind or have low vision, using color only to create emphasis can get lost. There are extra steps you can take to achieve good design, and I'll give you a hint: It's shapes. I'm searching for a new game to play in Apple Arcade.

Sneaky Sasquatch looks like fun. In the App Store, buttons use the color system blue, and some use shapes to help them stand out...

like the "start playing" button, which is using color, and it has a shape around it.

This really helps it stand out, and lets a viewer know it's not just a standard label. The extra visualization captures my attention and draws me in, but I'll have to save this game for later. Let's talk about another example. Although tab bars only use color by default to indicate the active tab, you can enable the Button Shapes accessibility setting to get

additional visualizations. If you want your buttons to really stand out while the button shapes setting is enabled, we're bringing new API to iOS 14 to help you accomplish this.

If you can't fit button shapes in your default design, you should provide an alternate appearance for people with this setting enabled.

To check this setting, you can call `button-Shapes-Enabled` on `UIAccessibility`, or you can observe the `button-Shapes-Enabled-Status-Did-Change-Notification` to run some code on the fly when this setting is changed. Last year in iOS 13, we added new API for `Differentiate without Color`, and this is a great way to extend these ideas beyond just buttons. You should apply this design practice to things like status icons, text with distinguishing colors, or anything else that relies on color alone to convey meaning. If you aren't sure where to start, try using SF Symbols.

We provide over 1,500 consistent, highly configurable symbols you can use in your app. There are so many to choose from, but if you don't find something that fits perfectly, you can create your own by expanding on icons provided by SF Symbols. Fun, right? SF Symbols are really powerful because they look great at any weight and scale. This means not only are you getting great symbolism, but you can configure the symbol to scale with text size and style. Let's dive in to some of my own UI, where I'm just getting started designing the visual experience with color and shapes. I'm working on a constellation tracking app, *Starstruck*.

Early stages of development are the best time to start thinking about visual accessibility accommodations, but it's really never too late to dig in. I have a list of zodiac constellations, and I'm distinguishing between them by choosing a custom color for each.

If you're color-blind, these might not stand out like I'd intend. Since I'm already working toward making these stand out, I don't want to stop at using just color, so I designed symbols to use in my app, giving each constellation a distinct visual experience by using color and shapes. This way, more people get to have a fun viewing experience when they use my app.

I'm also using bold text optionally to improve readability here, and I'll touch more on that later.

When we do use color in our apps, it's important to keep in mind that color contrast plays a huge role in readability, and it can be the difference between someone being able to see an element or having it completely blend in.

The Increase Contrast accessibility setting makes elements stand out by updating colors to high-contrast appearances. iOS offers a range of system colors that automatically adapt to settings like Increase Contrast.

If you're not using system colors, you'll need to be aware of this setting to improve the visual experience in your app. As a general rule, colors should get darker in Light Mode and lighter in Dark Mode. I know that might sound confusing, so here's an example.

In default contrast, my zodiac symbols are using colors I designed them with, but when Increase Contrast is turned on, I need something a little bit darker here. Now, I want you to keep an eye on the symbols in each cell as I enable Increase Contrast in settings.

Ready? Okay. I know that might have looked subtle, but this is a really big deal. For some people, having this boost in contrast can make a huge difference. If you are using custom

colors and symbols like this in your app, you'll need update the tint color or provide alternate assets for high-contrast appearances.

In my project's asset catalog, I can configure the appearance of my symbol by making changes in the Attributes Inspector. Under Appearances, you can check the High Contrast box to provide alternate versions of your symbols.

The High Contrast appearance will be used when Increase Contrast is on.

The Accessibility Inspector in Xcode has a super-handly Color Contrast Calculator.

You can use this tool to calculate the contrast between any two colors. Here, I'm calculating the contrast between my white symbol, which is in the foreground, against my custom purple color in the background.

This gives me a ratio of 4.5-to-1, which is generally the lowest acceptable ratio for most cases.

And here's my symbol's high-contrast appearance. I'm darkening the background color to really make that symbol pop, and my ratio is way better, at 7.5-to-1.

All right. Now that I've provided all of these variations, the work is done. iOS will automatically switch between them based on anyone's display preference.

And a shout-out to Smart Invert Colors, which, like Dark Mode, darkens bright white UI elements.

The difference is that Smart Invert Colors generally has higher contrast, so people with light sensitivity may prefer to use it. And whereas Dark Mode support in your apps is recommended, Smart Invert Colors is a system setting that asserts an inverted UI over any app. So it's great to keep in mind, someone using your app might prefer to use Smart Invert Colors.

You should specially flag certain views in your app so they don't get inverted, like photos, videos and app icons.

You can do that by setting accessibility-Ignores-Invert-Colors on any UIView subclass.

So when designing with color and shapes, you should take a variety of approaches to create visual emphasis.

Doing this adds a lot of character to your app-- Hello, branding-- while also making sure that the rich visual experience in your app is inclusive. And be sure to observe and respect people's preferences if your default design is not accommodating.

Next, I want to talk about text readability.

iOS provides support for changing the size of text on your device.

Smaller text can provide a higher content density by fitting more words on-screen, and large text can make things easy on the eyes.

Some people, however, rely on larger text sizes in order to read the content on their device.

When designing your app, keep text size, weight, and layout in mind for clarity and readability. My biggest tip is to think about these early on in the design process. This is a

great way to ensure your app looks good, not only for every content size, but also across many devices with different display sizes.

So design with large text in mind. You should avoid truncating text as the content size increases so that the user doesn't miss out on anything. Instead, wrap labels and use all of the available display width.

Remember those symbols I talked about earlier? Well, those also should scale with text.

You can override the function `traitCollectionDidChange`, which gets called when display traits are adjusted on the device.

And `UITraitCollection` lets you get the device's preferred content size category.

And, yes, you can use a comparison operator to perform comparisons on this enum. It's my favorite thing ever. In *Starstruck*, when the user changes their text size, I want to calculate whether it's a default font size or a large accessibility font size. For default font sizes, my stack views containing the cell's symbol and label are set to the horizontal axis with center alignment.

When an accessibility font size is enabled, I'm flipping the axis and choosing leading alignment. This lets my label take advantage of the full width of the display instead of sharing some of it with the symbol.

I'm also always setting this label's line count to zero instead of one, so that if it were longer, it would wrap for as long as it needs.

Remember how I mentioned that I'm using bold text optionally to improve readability? Using Bold Text not only lets labels stand out when they're up against symbols, but also lets them stand out when they're with other text as well. Did I mention *Starstruck* lets users geek out about star stats? Here, I'm using bold text optionally to create visual distinction between the title and detail label.

Creating emphasis with bold text is awesome. In fact, system-provided font styles lets you really change up your text weight, which means we handle the heavy lifting, and you don't need to lift a finger.

The Bold Text accessibility setting is for people that need this emphasis on all text in the system.

Sometimes thin and even default font styles are hard to read.

If you aren't using system font styles for text labels in your app, then it may not respond well to the Bold Text setting. You can check the setting by calling `isBoldTextEnabled` on `UIAccessibility` and make updates to your text styles manually.

Last, I want to talk about a few other display preferences which may alter the appearance of your app.

In my app, I've set up a parallax effect on my constellation view controller.

A parallax effect is a `UIMotionEffect` that helps me create a perception of depth between stars in the foreground and the vastness of the universe in the background.

As I gently move my device, the stars in the foreground shift against the artwork in the back.

Another example of parallax you might be familiar with is on the Home Screen. These effects are a lot of fun to use and experience, but for some people, this can be disorienting and cause motion sickness. There is a collection of display accommodation settings that people can enable if they're sensitive to motion.

While these display settings are on, the user will expect adjustments to motion on their device, like suppressing small idle animations, parallax or other motion effects, auto-playing videos or GIFs, and even slide transitions. We love using effects like this to bring the software experience to life, but it's also our duty to listen and respect the setting for users that have it enabled. If you are using dramatic or frequent animations in your app, check `isReduceMotionEnabled` on `UIAccessibility` before performing them.

And you should observe the `reduceMotionStatusDidChangeNotification` to make adjustments if the setting changes while your app is open.

I want to tell you a little more about the Reduce Motion setting `PreferCrossFadeTransitions`.

As I move through these navigation controllers, push segues are performed with sliding animations.

The next view controller slides into place to cover the previous one.

When the `PreferCrossFadeTransition` setting is on, we replace the sliding transitions for something a little more subtle.

And now you can too. In iOS 14, we are introducing new API for the `PreferCrossFadeTransition` setting.

If you're already using `UIKit`, `UINavigationController`, this work already comes for free.

But if you've implemented custom slide transitions in your app, this API is for you.

You can check if you need to cross-fade your transitions by calling `prefersCrossFadeTransitions` on `UIAccessibility` or observing the notification. Last is the `ReducedTransparency` setting, which sets blur effects to become completely opaque. One big benefit of this is text readability, because a blurry background might have varying levels of contrast, making things hard to read. While my users are geeking out about star stats, I added a visual effect to blur some artwork in the background.

This helps me create an experience that is immersive and feels unique, but this isn't suitable for everyone. Some blur effects can cause the background to have varying levels of contrast. When up against text, some readers will have a hard time seeing this. Or it can make them feel dizzy. So when `ReducedTransparency` is on, system blur effects and vibrancy effects are adjusted to become a single solid color. For some, this makes it much easier to read.

If you're not using system visual effects for blurs or vibrancy, `UIAccessibility` also provides API free to use to check the display preference for `ReducedTransparency`.

By accommodating these display settings, you can be sure your app looks great on anyone's device.

All of this is so important because being able to use an app that is inclusive and accommodating is, simply put, empowering for our customers.

Thank you for all of your work in making your app accessible for everyone. Now, I'm inviting you to go in accessibility settings, find some of the settings I talked about today and turn them on, so you can see what your app looks like. You'll be surprised by what already looks great, and this will be the perfect place to start figuring out what you can work on next.

Thanks, everyone.

- - Copy Code

3:14 - [Button Shapes](#)

```
func observeButtonShapesNotification() {
    // Make buttons more visible by using shapes.
    // If your default design does not include button shapes, observe
    NotificationCenter.default.addObserver(self, selector: #selector(select))
}

@objc func updateButtonShapes() {
    if UIAccessibility.buttonShapesEnabled {
        // Use extra visualizations for buttons.
    } else {
        // Use default design for buttons.
    }
}
```

- Copy Code

3:31 - [Differentiate Without Color](#)

```
func observeDifferentiateWithoutColorNotification() {
    // Use symbols or shapes to convey meaning instead of relying
    // If your default design does not differentiate without color
    NotificationCenter.default.addObserver(self, selector: #selector(select))
}

@objc func updateColorAndSymbols() {
    if UIAccessibility.shouldDifferentiateWithoutColor {
        // Use symbols or shapes to convey meaning.
    } else {
        // Use default design.
    }
}
```

- Copy Code

7:47 - [Smart Invert Colors](#)

```
extension UIView {
    @available(iOS 11.0, tvOS 11.0)
    var accessibilityIgnoresInvertColors: Bool { get set }
}
```

- Copy Code

9:57 - [Large Text](#)

```
// ZodiacConstellationCell.swift
```

```
override func traitCollectionDidChange (_ previousTraitCollection:
    if (traitCollection.preferredContentSizeCategory
        < .accessibilityMedium) { // Default font sizes

        stackView.axis = .horizontal
        stackView.alignment = .center

    } else { // Accessibility font sizes

        stackView.axis = .vertical
        stackView.alignment = .leading

    }
}
```

◦  Copy Code

11:33 - [Bold Text](#)

```
func observeBoldTextNotification() {
    // Update labels to use bold or heavy font styles.
    // If you aren't using system font styles, observe this notification
    NotificationCenter.default.addObserver(self, selector: #select
}

@objc func updateLabelWeight() {
    if UIAccessibility.isBoldTextEnabled {
        // Use bold or heavy font weight
    } else {
        // Use font weight that is default to your design.
    }
}
```

◦  Copy Code

13:08 - [Reduce Motion](#)

```
func observeReduceMotionNotification() {
    // Observe this notification to reduce or remove the frequency
    NotificationCenter.default.addObserver(self, selector: #select
}

@objc func updateMotionEffects() {
    if UIAccessibility.isReduceMotionEnabled {
        // Reduce or remove extraneous motion effects.
    } else {
        // Use default motion effects.
    }
}
```

```
}  
}
```

◦

Copy Code

13:51 - [Prefers Cross-fade Transitions](#)

```
func observeCrossFadeTransitionsNotification() {  
    // Reduce or remove sliding animations for transitioning views  
    // If you aren't using system-provided navigation, observe this  
    NotificationCenter.default.addObserver(self, selector: #selector(  
}  
  
@objc func updateTransitionEffects() {  
    if UIAccessibility.prefersCrossFadeTransitions {  
        // Replace sliding transitions with cross-fade animations.  
    } else {  
        // Use default sliding transitions.  
    }  
}
```

◦

Copy Code

15:07 - [Reduce Transparency](#)

```
func observeReduceTransparencyNotification() {  
    // Reduce or remove transparency by adjusting these effects to  
    // If you aren't using system-provided visual effects for blur  
    NotificationCenter.default.addObserver(self, selector: #selector(  
}  
  
@objc func updateTransparencyEffects() {  
    if UIAccessibility.isReduceTransparencyEnabled {  
        // Make transparency effects opaque.  
    } else {  
        // Use default transparency.  
    }  
}
```

Developer Footer

- [Videos](#)
- [WWDC20](#)
- Make your app visually accessible

Platforms

[Open Menu](#) [Close Menu](#)

- [iOS](#)

- [iPadOS](#)
- [macOS](#)
- [tvOS](#)
- [visionOS](#)
- [watchOS](#)
- [App Store](#)



Tools

[Open Menu](#) [Close Menu](#)

- [Swift](#)
- [SwiftUI](#)
- [Swift Playground](#)
- [TestFlight](#)
- [Xcode](#)
- [Xcode Cloud](#)
- [Icon Composer](#)
- [SF Symbols](#)



Technologies

[Open Menu](#) [Close Menu](#)

- [Accessibility](#)
- [Accessories](#)
- [Apple Intelligence](#)
- [Audio & Video](#)
- [Augmented Reality](#)
- [Business](#)
- [Design](#)
- [Distribution](#)
- [Education](#)
- [Games](#)
- [Health & Fitness](#)
- [In-App Purchase](#)
- [Localization](#)
- [Maps & Location](#)
- [Machine Learning & AI](#)
- [Security](#)
- [Safari & Web](#)



Resources

[Open Menu](#) [Close Menu](#)

- [Documentation](#)
- [Downloads](#)

- [Sample Code](#)
- [Videos](#)



Support

[Open Menu](#) [Close Menu](#)

- [Help Guides & Articles](#)
- [Contact Us](#)
- [Forums](#)
- [Feedback & Bug Reporting](#)
- [System Status](#)



Account

[Open Menu](#) [Close Menu](#)

- [Apple Developer](#)
- [App Store Connect](#)
- [Certificates, IDs, & Profiles](#)
- [Feedback Assistant](#)



Programs

[Open Menu](#) [Close Menu](#)

- [Apple Developer Program](#)
- [Apple Developer Enterprise Program](#)
- [App Store Small Business Program](#)
- [MFi Program](#)
- [Mini Apps Partner Program](#)
- [News Partner Program](#)
- [Video Partner Program](#)
- [Security Bounty Program](#)
- [Security Research Device Program](#)



Events

[Open Menu](#) [Close Menu](#)

- [Meet with Apple](#)
- [Apple Developer Centers](#)
- [App Store Awards](#)
- [Apple Design Awards](#)
- [Apple Developer Academies](#)
- [WWDC](#)

Read the [latest news](#) Get the [Apple Developer app](#).



Light



Dark



Auto

V

Copyright © 2026 [Apple Inc.](#) All rights reserved.
[Terms of Use](#) [Privacy Policy](#) [Agreements and Guidelines](#)